Neural Networks in PyTorch

Lecture 7: Neural networks, Parts 7a and 7b

Khiem Nguyen

28 October 2026
James Watt School of Engineering, University of Glasgow

Table of contents

1 Objectives 1

2 Tools 2

3 From the formulas to PyTorch 2
3.1 Forward propagation . . . 2
3.2 Backpropagation is what autograd does . . . 3
3.3 Activation functions . . . 3

4 The MNIST digits 4

5 A network from plain tensors 5
5.1 The cross-entropy, three ways . . . 5
5.2 Gradients accumulate . . . 6
5.3 Training . . . 6

6 The lecture's network as an `nn.Module` 7
6.1 Training and test loops . . . 7
6.2 Mistakes . . . 8

7 Challenges 9

8 Summary 9

1 Objectives

The previous two tutorials built networks from scratch in NumPy. PyTorch provides the same building blocks, and computes the backward pass for us. In this tutorial you will

- check that `nn.Linear` and `nn.Sequential` compute exactly the forward-propagation formulas of the lecture;
- check that `loss.backward()` computes exactly the gradients of our NumPy backpropagation;
- load the MNIST digits with `torchvision` and a `DataLoader`;
- train a network written with plain tensor operations, and then the lecture's network written as an `nn.Module`;

- evaluate the trained network on the test set.

2 Tools

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"PyTorch {torch.__version__}, running on {device}")
```

```
PyTorch 2.13.0+cu132, running on cuda
```

3 From the formulas to PyTorch

3.1 Forward propagation

`nn.Linear(n_in, n_out)` stores a weight matrix of shape `(n_out, n_in)`, exactly like $\mathbf{W}^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$ in the lecture, and computes `X @ W.T + b` for a batch `X` with one example per row. `nn.Sequential` applies its layers in order, which is forward propagation. We build the lecture's $3 \to 4 \to 1$ network and compare it with the formulas:

```
torch.manual_seed(0)
model = nn.Sequential(
    nn.Linear(3, 4),   # W1: (4, 3), b1: (4,)
    nn.Sigmoid(),
    nn.Linear(4, 1),   # W2: (1, 4), b2: (1,)
    nn.Sigmoid(),
)
W1, b1 = model[0].weight, model[0].bias
W2, b2 = model[2].weight, model[2].bias

X = torch.rand(5, 3)                          # m = 5 examples, d = 3 features
A1 = torch.sigmoid(X @ W1.T + b1)             # (5, 4)
A2 = torch.sigmoid(A1 @ W2.T + b2)            # (5, 1)
print(f"W1 shape {tuple(W1.shape)}, W2 shape {tuple(W2.shape)}")
print(f"same output as model(X): {torch.allclose(A2, model(X))}")
print(f"number of parameters: {sum(p.numel() for p in model.parameters())}")
```

```
W1 shape (4, 3), W2 shape (1, 4)
same output as model(X): True
number of parameters: 21
```

The network has $4 \cdot (3+1) + 1 \cdot (4+1) = 21$ parameters.

3.2 Backpropagation is what autograd does

In the first tutorial we wrote backpropagation by hand. PyTorch's **autograd** records every operation of the forward pass and, when we call `loss.backward()`, applies the chain rule backwards through them. On the same network and the same data, the two must agree. Here is the lecture's NumPy backpropagation for a sigmoid hidden layer and a sigmoid output:

```
def backprop_numpy(X, y, W1, b1, W2, b2):
    """Gradients of the binary cross-entropy for a 2-layer sigmoid network
    ↪ (NumPy, rows = examples)."""
    sigmoid = lambda z: 1 / (1 + np.exp(-z))
    m = X.shape[0]
    A1 = sigmoid(X @ W1.T + b1)                  # forward
    A2 = sigmoid(A1 @ W2.T + b2)
    dZ2 = A2 - y                                 # backward
    dW2, db2 = dZ2.T @ A1 / m, dZ2.mean(axis=0)
    dZ1 = (dZ2 @ W2) * A1 * (1 - A1)
    dW1, db1 = dZ1.T @ X / m, dZ1.mean(axis=0)
    return [dW1, db1, dW2, db2]

model = model.double()                           # float64, to compare with NumPy
↪ precisely
X = torch.rand(8, 3, dtype=torch.float64)        # m = 8 examples
y = torch.randint(0, 2, (8, 1)).double()

loss = nn.BCELoss()(model(X), y)                 # binary cross-entropy, averaged
↪ over the batch
loss.backward()                                  # autograd: fills p.grad for
↪ every parameter
grads_torch = [model[0].weight.grad, model[0].bias.grad, model[2].weight.grad,
↪ model[2].bias.grad]

params_np = [p.detach().numpy() for p in (model[0].weight, model[0].bias,
↪ model[2].weight, model[2].bias)]
grads_np = backprop_numpy(X.numpy(), y.numpy(), *params_np)
diff = max(np.max(np.abs(g_np - g_t.numpy())) for g_np, g_t in zip(grads_np,
↪ grads_torch))
print(f"largest difference between NumPy and autograd: {diff:.1e}")
```

```
largest difference between NumPy and autograd: 2.8e-17
```

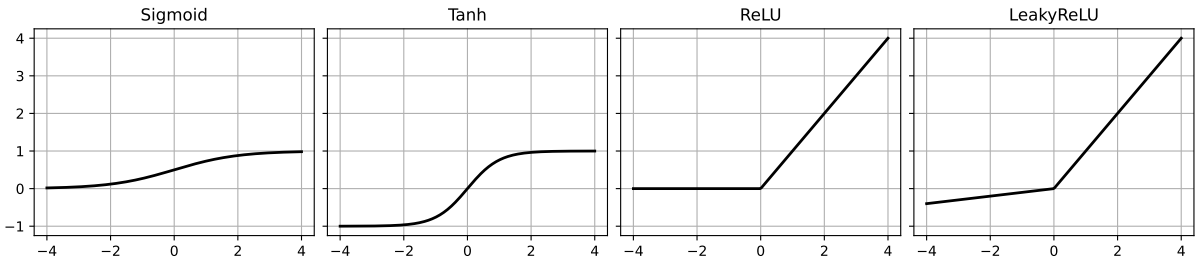
The difference is pure rounding error. From now on we only write the **forward** pass, and autograd derives the backward pass.

3.3 Activation functions

PyTorch provides each activation function both as a layer (e.g. `nn.ReLU()`, for use in `nn.Sequential`) and as a function (e.g. `torch.relu` or `F.relu`):

```
z = torch.linspace(-4, 4, 200)
layers = [nn.Sigmoid(), nn.Tanh(), nn.ReLU(), nn.LeakyReLU(negative_slope=0.1)]
fig, ax = plt.subplots(1, 4, figsize=(12, 2.6), sharey=True,
↪ constrained_layout=True)
for a, layer in zip(ax, layers):
```

```
    a.plot(z, layer(z), 'k-', lw=2)
    a.set_title(layer.__class__.__name__)
    a.grid(True)
plt.show()
print(f"nn.ReLU() and torch.relu agree: {torch.equal(nn.ReLU()(z),
 ↪  torch.relu(z))}")
```




```
nn.ReLU() and torch.relu agree: True
```

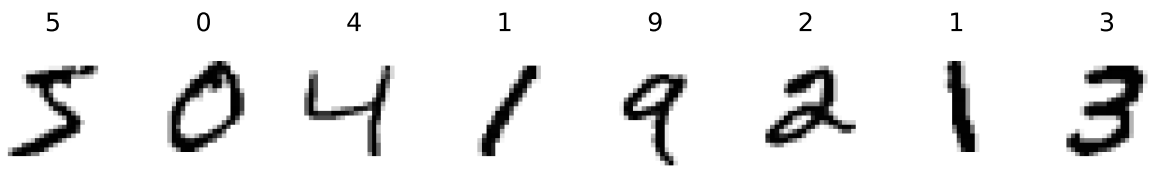
4 The MNIST digits

MNIST contains 70 000 greyscale images of handwritten digits, 28×28 pixels each: 60 000 for training and 10 000 for testing. `torchvision` downloads them once into the folder `data`. The transform `ToTensor()` turns each image into a tensor of shape `(1, 28, 28)` with values in $[0, 1]$.

```
train_data = datasets.MNIST("data", train=True, download=True,
 ↪  transform=ToTensor())
test_data = datasets.MNIST("data", train=False, download=True,
 ↪  transform=ToTensor())
image, label = train_data[0]                    # one example: (image, label)
print(f"{len(train_data)} training and {len(test_data)} test images; "
      f"one image has shape {tuple(image.shape)} and label {label}")

fig, ax = plt.subplots(1, 8, figsize=(10, 1.6))
for a, i in zip(ax, range(8)):
    image, label = train_data[i]
    a.imshow(image[0], cmap='gray_r')
    a.set_title(f'{label}')
    a.axis('off')
plt.show()
```

```
60000 training and 10000 test images; one image has shape (1, 28, 28) and label 5
```



A `DataLoader` serves the data in shuffled mini-batches, here of 64 images:

```
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64)

images, labels = next(iter(train_loader))        # the first batch
print(f"images: {tuple(images.shape)}, labels: {tuple(labels.shape)}")
```

```
images: (64, 1, 28, 28), labels: (64,)
```

A batch of images has shape `(64, 1, 28, 28)`: batch, colour channel, height and width. A fully connected network needs each image as one row of 784 pixel values, so we will flatten it to shape `(64, 784)`.

5 A network from plain tensors

To see what the layers of `torch.nn` do, we first write a network $784 \to 128 \to 10$ with plain tensor operations. The parameters are tensors with `requires_grad=True`, so autograd tracks them. The 10 outputs are the scores of the digits 0 to 9, the **logits**; softmax turns them into probabilities (Lecture 3b).

```
torch.manual_seed(0)
n_in, n_hidden, n_out = 784, 128, 10
W1 = (torch.randn(n_hidden, n_in) * np.sqrt(2 / n_in)).requires_grad_()   # He
 ↪  initialisation
b1 = torch.zeros(n_hidden, requires_grad=True)
W2 = (torch.randn(n_out, n_hidden) * np.sqrt(1 / n_hidden)).requires_grad_()
b2 = torch.zeros(n_out, requires_grad=True)
parameters = [W1, b1, W2, b2]

def net(images):
    """Forward pass: flatten, then 784 -> 128 (ReLU) -> 10 logits."""
    X = images.reshape(-1, n_in)                  # (m, 784)
    A1 = torch.relu(X @ W1.T + b1)                # (m, 128)
    return A1 @ W2.T + b2                         # (m, 10) logits
```

5.1 The cross-entropy, three ways

The cross-entropy of Lecture 3b can be computed from the softmax probabilities and one-hot labels, but PyTorch's `nn.CrossEntropyLoss` and `F.cross_entropy` take the **logits** directly and combine softmax and logarithm in one numerically stable step. All three give the same value:

```
logits = net(images)
probs = torch.softmax(logits, dim=1)
Y = F.one_hot(labels, num_classes=10)            # (64, 10)
loss_by_hand = -torch.mean(torch.sum(Y * torch.log(probs), dim=1))
print(f"by hand:             {loss_by_hand.item():.6f}")
print(f"nn.CrossEntropyLoss: {nn.CrossEntropyLoss()(logits, labels).item():.6f}")
print(f"F.cross_entropy:     {F.cross_entropy(logits, labels).item():.6f}")
```

```
by hand:             2.382397
nn.CrossEntropyLoss: 2.382397
F.cross_entropy:     2.382397
```

Never apply softmax before `nn.CrossEntropyLoss`: it expects logits, and applies softmax itself.

5.2 Gradients accumulate

`loss.backward()` **adds** the new gradient to the `.grad` attribute of every parameter. If we forget to reset it, the gradients of successive batches pile up:

```
for call in range(1, 4):
    loss = F.cross_entropy(net(images), labels)   # new forward pass on the same
 ↪  batch
    loss.backward()
    print(f"after backward call {call}: |dJ/db2| = {b2.grad.norm().item():.4f}")
for p in parameters:
    p.grad = None                                 # reset, as
 ↪  optimizer.zero_grad() does
```

```
after backward call 1: |dJ/db2| = 0.1414
after backward call 2: |dJ/db2| = 0.2828
after backward call 3: |dJ/db2| = 0.4241
```

The norm grows linearly with the number of calls, which is why the training loop calls `optimizer.zero_grad()` before every backward pass.

5.3 Training

The training loop repeats, for every mini-batch: forward pass and loss, reset the gradients, backward pass, and an update step. `torch.optim.SGD` performs the gradient-descent update $\theta \leftarrow \theta - \alpha\, \partial J/\partial \theta$ for all parameters it was given.

```
def accuracy(model_fn, loader):
    """Fraction of correctly classified examples."""
    correct = 0
    with torch.no_grad():                         # no gradients needed for
     ↪  evaluation
        for images, labels in loader:
            correct += (model_fn(images).argmax(dim=1) == labels).sum().item()
    return correct / len(loader.dataset)

optimizer = torch.optim.SGD(parameters, lr=0.1)
for epoch in range(3):
    for images, labels in train_loader:
        loss = F.cross_entropy(net(images), labels)   # forward pass and loss
        optimizer.zero_grad()                         # reset the gradients
        loss.backward()                               # backpropagation
        optimizer.step()                              # gradient-descent update
    print(f"epoch {epoch + 1}: test accuracy {accuracy(net, test_loader):.1%}")
```

```
epoch 1: test accuracy 93.2%
epoch 2: test accuracy 94.8%
epoch 3: test accuracy 95.9%
```

After three epochs, this small network already recognises about 96% of the test digits.

6 The lecture's network as an `nn.Module`

In practice we define a model as a class that inherits from `nn.Module`: the layers are created in `__init__`, and the computation is written in `forward`. PyTorch then finds all parameters automatically. This is the lecture's network $784 \to 512 \to 512 \to 10$:

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()                  # (batch, 1, 28, 28) ->
        ↪  (batch, 784)
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28 * 28, 512), nn.ReLU(),
            nn.Linear(512, 512), nn.ReLU(),
            nn.Linear(512, 10),                      # 10 logits, one per digit
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.linear_relu_stack(x)

torch.manual_seed(0)
model = NeuralNetwork().to(device)
print(model)
print(f"number of parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
number of parameters: 669,706
```

Only the linear layers have parameters: $784 \cdot 512 + 512$, $512 \cdot 512 + 512$ and $512 \cdot 10 + 10$, in total 669 706.

6.1 Training and test loops

We put one epoch of training and the evaluation on the test set into two functions, as in the lecture. The batches are moved to the same device as the model, which matters when a GPU is available.

```
def train_loop(loader, model, loss_fn, optimizer):
    model.train()
    for X, y in loader:
        X, y = X.to(device), y.to(device)
        loss = loss_fn(model(X), y)          # forward pass and loss
        optimizer.zero_grad()
```

```
        loss.backward()                     # backpropagation
        optimizer.step()                    # gradient-descent update

def test_loop(loader, model, loss_fn):
    model.eval()
    correct, total_loss = 0, 0.0
    with torch.no_grad():                   # no gradients needed for evaluation
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            total_loss += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(dim=1) == y).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n      # average loss, accuracy

loss_fn = nn.CrossEntropyLoss()             # softmax + cross-entropy, from
 ↪ logits
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for epoch in range(10):
    train_loop(train_loader, model, loss_fn, optimizer)
    test_loss, test_acc = test_loop(test_loader, model, loss_fn)
    print(f"epoch {epoch + 1:2d}: test loss {test_loss:.3f}, accuracy
     ↪ {test_acc:.1%}")
```

```
epoch  1: test loss 0.214, accuracy 93.5%
epoch  2: test loss 0.132, accuracy 96.2%
epoch  3: test loss 0.100, accuracy 96.8%
epoch  4: test loss 0.092, accuracy 97.0%
epoch  5: test loss 0.145, accuracy 95.4%
epoch  6: test loss 0.080, accuracy 97.6%
epoch  7: test loss 0.070, accuracy 97.7%
epoch  8: test loss 0.065, accuracy 97.9%
epoch  9: test loss 0.096, accuracy 96.7%
epoch 10: test loss 0.061, accuracy 98.0%
```

The test accuracy reaches about 98%. It does not increase at every epoch, because each step of mini-batch gradient descent uses only 64 examples and is therefore noisy.

`model.train()` and `model.eval()` switch layers such as dropout between training and evaluation mode. Our network has no such layers, so they change nothing here, but calling them is good practice.

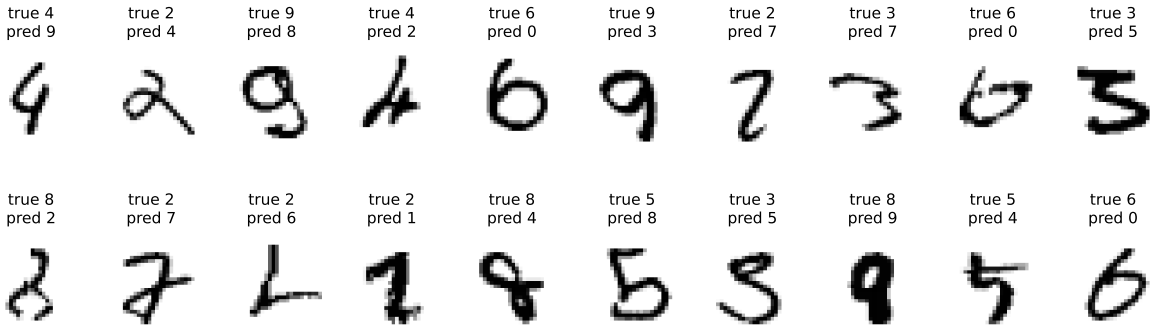
6.2 Mistakes

Which test images does the network get wrong? We predict the whole test set and show some of the misclassified digits:

```
model.eval()
with torch.no_grad():
    preds = torch.cat([model(X.to(device)).argmax(dim=1).cpu() for X, _ in
 ↪ test_loader])
targets = test_data.targets
wrong = torch.nonzero(preds != targets).flatten()
print(f"{len(wrong)} of {len(targets)} test images are misclassified")
```

```
fig, ax = plt.subplots(2, 10, figsize=(11, 3.2))
for a, i in zip(ax.flat, wrong[:20]):
    a.imshow(test_data[i][0][0], cmap='gray_r')
    a.set_title(f'true {targets[i].item()}\npred {preds[i].item()}', fontsize=8)
    a.axis('off')
plt.show()
```

```
198 of 10000 test images are misclassified
```



Many of these digits are hard to read even for a human.

7 Challenges

1. Train the `nn.Module` network with learning rates 0.01 and 1.0. What happens?
2. Replace SGD by `torch.optim.Adam(model.parameters(), lr=1e-3)`. Then try Adam with `lr=0.05`. What do you observe?
3. Logistic regression on MNIST is a network without hidden layer: `nn.Sequential(nn.Flatten(), nn.Linear(784, 10))`. Train it in the same way. How much do the hidden layers gain?
4. How many parameters does the network $784 \to 25 \to 1$ from the lecture (digit 0 versus 1) have? Check your answer with PyTorch.

8 Summary

- `nn.Linear(n_in, n_out)` stores $\mathbf{W}$ with shape `(n_out, n_in)` and computes `X @ W.T + b`; `nn.Sequential` is forward propagation.
- `loss.backward()` is backpropagation: autograd gives the same gradients as our NumPy code. Gradients accumulate, so call `optimizer.zero_grad()` before every backward pass.
- `nn.CrossEntropyLoss` takes logits and combines softmax and cross-entropy.
- A model is an `nn.Module` with layers in `__init__` and the computation in `forward`; training repeats forward, loss, `zero_grad`, `backward` and `step` for every mini-batch.
- The lecture's network reaches about 98% test accuracy on MNIST after 10 epochs.